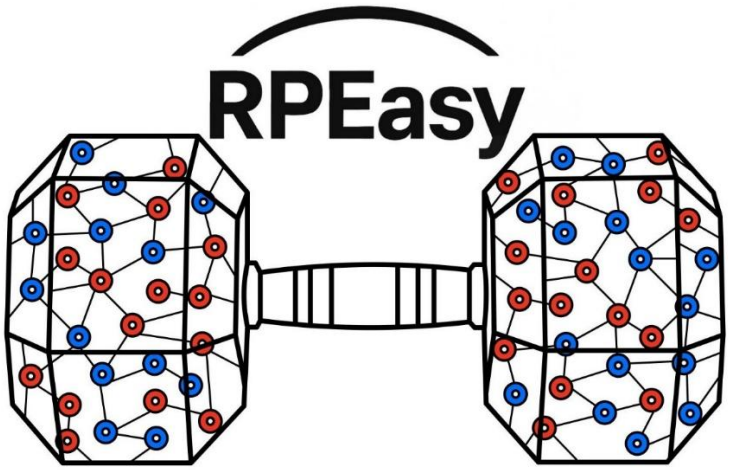
ELEC 490/498

Final Report



Project title: RPEasy

Group #: 53

Group members: 1) Luke Hunker (20lrh5) (20287422)

2) Avedis Boudakian (20ajb27) (20287296)

3) Ajmal Niazi (21an18) (20300697)

4) Rhys Blakeney (20wrb1) (20283269)

Email to faculty supervisor(s): Karen Rudie

Submission date: 04/07/2026

Executive Summary

The Rate of Perceived Exertion (RPE) is a foundational metric in exercise science used to measure the strenuousness of physical activity. For weightlifting, achieving a high RPE (8-10) is critical for effective muscle growth. However, beginner lifters frequently misjudge their exertion levels, leading to suboptimal training stimulus or the risk of overtraining. To address this, RPEasy was created. It is an accessible, software-based training tool that utilizes computer vision and machine learning to objectively predict an athlete's RPE based on physiological factors.

The foundation of the RPEasy system is a proprietary dataset of over 750 weightlifting videos, consisting of squats, bench presses, and deadlifts, sourced primarily from real-world, gym environments. To eliminate subjective bias and establish an accurate ground truth, a certified Level 5 GoodLife personal trainer was hired to manually review and label the RPE of every video.

The system's backend pipeline employs a multi-modal computer vision approach to extract three key biomechanical features linked to physical exertion. First, facial strain is analyzed using OpenFace, guided by a MediaPipe preprocessor, to extract the maximum intensity of specific Facial Action Units (e.g., brow lowering). Second, bar speed tracking is executed using a hybrid MMPose and OpenCV optical flow approach to calculate velocity loss across a set. Finally, form breakdown is quantified using MMPose by calculating a confidence weighted difference metric between the first and last repetitions of a lift.

These extracted features are aggregated and fed into a machine-learning regression model tasked with estimating the final RPE. To prevent overfitting on the dataset, the project

utilizes a gradient-boosting ensemble architecture consisting of a 60% LightGBM and 40% XGBoost split. Athletes interact with the backend via a custom, cross-platform mobile application built with Flutter, which asynchronously processes user-uploaded videos and translates the machine-learning arrays into readable, color-coded training insights.

Rigorous testing validated the effectiveness of the RPEasy pipeline. The final LightGBM and XGBoost ensemble achieved a Mean Absolute Error (MAE) of 0.67 on a 1-10 scale, successfully demonstrating that the synthesis of bar speed change, form breakdown, and facial strain can objectively quantify physiological fatigue. This is a highly viable and accurate predictive baseline, as confirmed by domain experts. Feature importance analysis revealed that the product of the form difference metric and repetition tempo variability was the heaviest driver of the prediction, proving the necessity of the multi-modal approach.

The project was completed under budget, utilizing $480.25 of the $600.00 allowance entirely for professional data labeling, as hardware costs were eliminated by leveraging existing mobile devices. Ultimately, the project highlighted the resource-intensive costs of data engineering and the limitations of rigid analysis frameworks like OpenFace in chaotic gym environments. For future iterations that aim for commercial scalability, migrating to a lightweight, deep learning model that can process facial strain natively on mobile devices, would be great for biometric privacy and eliminating cloud processing latency.

Table of Contents

1. Introduction 1
2. Design 2
2.1 Dataset Requirements and Strategy 2
2.2 Backend Pipeline 3
2.3 Computer Vision Feature Extraction 7
2.3.1 Facial Strain Extraction 7
2.3.2 Bar Speed Tracking 9
2.3.3 Form Detection 10
2.4 Model Design 11
2.5 Application UI 12
3. Implementation 14
3.1 Dataset Creation 14
3.2 Facial Strain Extraction- OpenFace 16
3.2.1 Pose-Guided Preprocessor Implementation 17
3.2.2 OpenFace Extraction 18
3.2.3 Imputation and Data Cleaning Implementation 18
3.3 Bar Speed Tracking- OpenCV 20
3.3.1 Initialization and Point Tracking 20
3.3.2 Dimensional Scaling and Calibration 21
3.3.3 Data Processing and Phase Segmentation 21
3.3.4 Feature Extraction for the Machine Learning Model 22
3.4 Form Detection - MMPose 22
3.4.1 General Motion Rules 23
3.4.2 Exercise-Specific Rules 24
3.4.3 Rep Detection and Difference Metric Calculation 26
3.5 Model Training 27
3.6 UI- RPEasy Application 30
3.6.1 Video Acquisition and Preprocessing Interface 30
3.6.2 Backend API Integration and State Management 30

3.6.3 Dynamic Data Rendering and Form Breakdown Insights 31
3.7 Final Project Budget 32
4. Testing 33
4.1 Facial Strain Extraction - OpenFace 33
4.1.1 Verification of Face Detection Reliability: 33
4.1.2 Landmark Stability 34
4.1.3 Facial AU Accuracy 34
4.2 Bar Speed Tracking- OpenCV 35
4.2.1 Tracking Robustness and Visual Reliability 35
4.2.2 Repetition Segmentation and Timing Accuracy 36
4.2.3 Fatigue Metric Δv Validation 37
4.3 Form Detection- MMPose 37
4.3.1 Tracking Accuracy 38
4.3.2 Difference Metric Validation 39
4.4 Model Validation and Testing 39
5. Results 41
6. Conclusion and Reflections 44
6.1 Project Conclusion 44
6.2 Reflections on Engineering and Data Challenges 44
6.3 Future Recommendations and Commercialization 45
6.4 Societal and Cultural Impacts 46
References 46
Appendix 49

Table of Figures

Figure 1: ML Training Pipeline Flow Chart 4
Figure 2: Master Excel File 5
Figure 3: Feature Engineering Code 6
Figure 4: RPEasy UI 14
Figure 5: Convert File Type to MP4 Function 15
Figure 6: OpenFace Processing Pipeline 17
Figure 7: Poseguided Face Detector Code for Obtaining Crop Region Via MediaPipe 18
Figure 8: Code Block for Running OpenFace C++ Binary Via Subprocess 18
Figure 9: Feature Extracting Isolating for Maximum Exertion AU Values 19
Figure 10: Code Block for Aggregating Missing Data Points 19
Figure 11: Data Aggregation Pipeline 20
Figure 12: Jump Clamping Function for Squats 23
Figure 13: Motion Smoothing Function 24
Figure 14: Arm Separation Function 25
Figure 15: Elbow Level Function 25
Figure 16: Forearm Parallelism Function 26
Figure 17: Confidence-Weighted Difference Metric Function 27
Figure 18: LGBMMod2_Ensemble.py Libraries 27
Figure 19: Model Hyperparameters 28
Figure 20: LGBMMod2_Ensemble.py Training Loop 29
Figure 21: LGBMMod2_Ensemble.py Model Weight Saving 29
Figure 22: OpenFace Visualization 34
Figure 23: Bar Speed Tracking Visualization 36
Figure 24: Key point Visualization for Squat 38
Figure 25: Ensemble Predictions 40
Figure 26: Model Comparison 40
Figure 27: LGBM Feature Importance Ranking 42

Table of Tables

Table 1: Explanation of important features 42
Table 2: Project requirements from blueprint 43

1. Introduction

The rate of perceived exertion (RPE) scale is an established metric in exercise science that effectively measures how strenuous an exercise feels to the person performing it. For weightlifting, achieving an RPE of 8-10 (close to failure) is one of the most important factors for effective training stimulus leading to muscle growth. Research shows that beginner lifters can often underestimate their capabilities, self-assessing a higher RPE than they actually achieved, resulting in less effective training [17]. Furthermore, metrics like bar speed and form breakdown have been experimentally identified as useful metrics for predicting how close a set is to failure [18]. Based on these factors, providing RPE feedback to lifters would be a useful training tool for beginner lifters to safely increase their workout intensity for optimal training results. Furthermore, RPE feedback can be useful for all skill levels of lifters to ensure they are pushing themselves hard enough to provide effective muscle stimulus, but not over-exerting themselves resulting in overtraining.

This project's goal was to create an accessible software tool to provide RPE feedback to lifters such that they can make more informed training decisions. Traditionally, this type of feedback has been provided by a personal trainer or coach, but no such software-based tool currently exists. By using computer vision and machine learning, this project aims to leverage physiological indicators of RPE to create a tool which takes input videos of someone performing a lifting exercise, and outputs a predicted RPE score for that video [15][16].

2. Design

2.1 Dataset Requirements and Strategy

The development of RPEasy and the machine learning pipeline required a unique, robust, diverse, and accurately labeled dataset of weightlifting videos. A carefully designed and tailored dataset was needed to ensure the final LightGBM and XGBoost ensemble model could meet their performance goal of ±0.5 RPE. The computer vision subsystems (OpenFace, OpenCV, and MMPose) rely entirely on the integrity of this foundational dataset to extract reliable features.

To ensure the model generalizes well across the three core powerlifting movements, the dataset was designed to contain an even split of roughly 33% each for the squat, bench press, and deadlift. The next key design choice was sourcing the majority of the videos from online sources (YouTube, Instagram, TikTok, etc.) with the rest being videos recorded by the RPEasy team. Roughly 90% of the videos were from online sources and captured a wide variance of recording conditions. This allowed the model to be trained on real-world, suboptimal gym environments with varying recording angles, lighting conditions, and background noise. The final subset of videos was recorded by the project team and contained videos with ideal conditions (optimal camera angles, clear lighting, minimal noise). This subset was limited to 10% of the dataset to prevent the model from overfitting to the biomechanics of the development team and ideal environments.

For the computer vision feature extractors to process the data reliably, strict functional requirements were placed on the video files. First, all of the videos were required to be

standardized to the MP4 format to ensure pipeline compatibility and consistent frame-by-frame extraction. Second, to minimize noise, the videos had to start exactly when the first repetition started and end precisely at the completion of the final repetition, removing all un-racking, re-racking, and resting phases. Lastly, to eliminate uneducated bias and ensure a reliable ground truth RPE label for each video, all of the videos were independently reviewed and labeled by Trevor Felix, a certified level 5 personal trainer at GoodLife Fitness.

An AI project of this scale traditionally requires a massive dataset. However, the dataset for RPEasy was constrained to 750+ videos by strict project limitations regarding time, budget, and manual labor. To mitigate this size constraint, a data augmentation strategy was designed. By taking existing videos in the dataset and cropping out the final repetitions, the team could synthetically generate new videos representing lower RPE sets. For example, an 8-repetition set labeled as an RPE 9 could be cropped to 7 repetitions, creating a new video sample with a new RPE of 7 or 8. This allowed the team to artificially expand the dataset's volume and RPE variance without requiring additional data collection.

2.2 Backend Pipeline

The RPEasy system flows through its automated backend training pipeline which takes a complete dataset of videos and creates a fully trained model. This multi-stage pipeline was designed to transform visual information into structured features for machine learning, while addressing constraints in data volume, processing time, and expert availability. The high-level flow of this pipeline is illustrated in Figure 1.

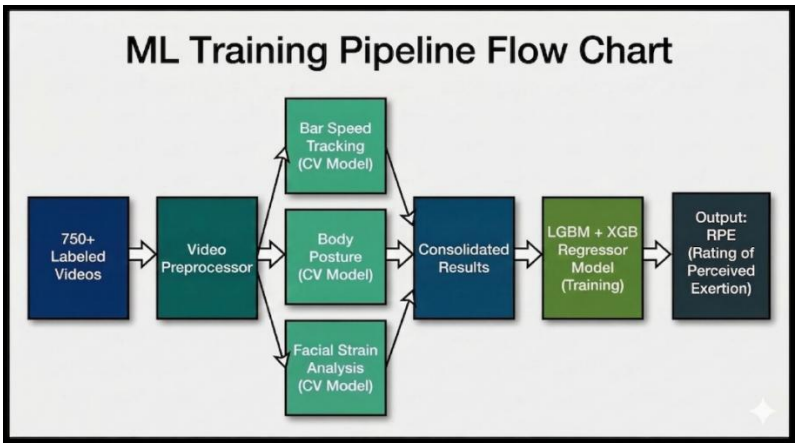


Figure 1: ML Training Pipeline Flow Chart

To process 750+ varied videos reliably, extract complex biomechanical features, and train an ensemble model, a video preprocessor python script was applied first to each video. This preprocessor isolated the primary lifter and removed jitter to allow a stable consistent video using MediaPipe. Additionally, to minimize manual intervention, the pipeline was built as a series of Python operations which connect all of the feature extraction operations and model training into a single environment.

The project requires the extraction of three unique biomechanical features for RPE estimation: facial strain, bar speed, and skeletal pose. The Pipeline implements three different computer vision models (OpenFace, MMDetection, and MMPose) in parallel to extract these features. The extractors generate data per video frame and the raw extracted values lead to the curse of dimensionality phenomenon which causes overfitting during model training. Hence, when consolidating the feature extraction results, a python script is run on the raw data and selects only valuable information. As shown in the "master excel file" (Figure 2), this reduces the total feature count by focusing only on key metrics like the

max facial actuation unit across the set, bar speed change, and pose deviation. The script merges these features with the baseline RPE labels into a single file.

	A	B	C	D	E	F	M	N	O	P	Q	R	S	T	U	V	W
1	Set	RPE	rep_count	rep_change_s	1_rep_max	d_value	AU04_max	AU06_max	AU07_max	AU09_max	AU10_max	AU12_max	AU17_max	AU20_max	AU25_max	AU26_max	au_highly_active_count
2	Bench Press 1	7	3	0.515967014	FALSE	0.164687566	0.85	0	0.42	0.7	0.07	0.02	1.24	0.86	0.74	1.37	0
3	Bench Press 2	10	4	-0.116554529	FALSE	0.571723893	1.71	1.28	3.05	0.72	0.91	1.36	2.23	3.85	2.96	3.9	1
4	Bench Press 3	8	1	6.286806283	TRUE	4.373691116	1.506315789	1.110526316	2.222105263	0.991578947	1.576315789	0.967368421	3.828947368	2.327368421	4.057368421	2.947894737	0.842105263
5	Bench Press 4	9	1	8.403438114	TRUE	3.590047515	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
6	Bench Press 5	10	1	12.03475103	TRUE	2.832819718	1.21	1.05875	2.20125	1.32875	1.99875	1.1225	3.38375	2.33625	2.95375	3.315	0.75
7	Bench Press 6	7	4	0.416598916	FALSE	2.51503948	0.974444444	0.841111111	1.867777778	0.870555556	1.332222222	0.810555556	2.696666667	1.488333333	2.376111111	2.720555556	0.777777778
8	Bench Press 7	8	5	1.49981774	FALSE	0.571060685	1.53	0.69	0.46	0.37	0.62	1.18	4.46	3.58	1.95	1.61	1
9	Bench Press 8	9	4	1.533333333	FALSE	1.034193132	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
10	Bench Press 9	7	3	0.4	FALSE	0.203038124	0.974444444	0.841111111	1.867777778	0.870555556	1.332222222	0.810555556	2.696666667	1.488333333	2.376111111	2.720555556	0.777777778
11	Bench Press 10	10	1	1.2616	TRUE	3.248377821	1.21	1.05875	2.20125	1.32875	1.99875	1.1225	3.38375	2.33625	2.95375	3.315	0.75
12	Bench Press 11	10	2	1.199635674	FALSE	3.248377821	0.36	1.69	2.14	0.22	2.75	1.95	3.25	0.99	1.84	1.53	1
13	Bench Press 12	9	1	3.7	TRUE	5.759633241	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
14	Bench Press 13	6	2	0.199709571	FALSE	2.949087332	1.255	1.034	2.158	1.281	1.903	1.08	3.053	1.765	2.335	2.767	1.181818182
15	Bench Press 14	9	1	2.766666667	TRUE	5.759633241	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
16	Bench Press 15	7	4	1.599156886	FALSE	1.389780346	0.974444444	0.841111111	1.867777778	0.870555556	1.332222222	0.810555556	2.696666667	1.488333333	2.376111111	2.720555556	0.777777778
17	Bench Press 16	9	3	7.493023256	FALSE	1.649912052	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
18	Bench Press 17	6	3	0.49730198	FALSE	0.435763176	1.255	1.034	2.158	1.281	1.903	1.08	3.053	1.765	2.335	2.767	1.181818182
19	Bench Press 18	8	3	0.932655811	FALSE	0.475157079	0.1	0	0.33	0.69	1.35	0.59	1.52	0.48	3.4	1.43	0
20	Bench Press 19	9	1	2	TRUE	5.759633241	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
21	Bench Press 20	8	5	1.064357025	FALSE	1.121636649	1.506315789	1.110526316	2.222105263	0.991578947	1.576315789	0.967368421	3.828947368	2.327368421	4.057368421	2.947894737	0.842105263
22	Bench Press 21	7	13	0.499942922	FALSE	3.385429414	1.89	1.12	0.36	1.1	1.33	0.83	2.6	2.21	3.15	2.3	0
23	Bench Press 22	8	14	1.232288342	FALSE	3.197703736	2.09	0.91	0.54	1.13	1.6	1.14	2.29	2.01	3.28	2.53	0
24	Bench Press 23	10	15	1.832198697	FALSE	0.358049579	1.54	0.52	1.8	0.91	1.54	0.43	2.98	3.13	2.51	2.36	0
25	Bench Press 24	7	3	4.372150049	FALSE	2.470563762	2.54	0.6	1.67	1.1	1.33	0.49	4.27	1.07	5	3.09	1
26	Bench Press 25	8	5	0.399567319	FALSE	3.812472304	1.59	0.55	1.33	0.27	0.89	0.6	3.27	3.2	1.73	2.44	0
27	Bench Press 26	9	6	0.682642255	FALSE	3.908396285	2.27	0.38	0.88	0.4	0.91	0.34	2.4	3.32	3.11	5	1
28	Bench Press 27	9	3	0.765837838	FALSE	0.215475622	1.568461538	1.264615385	3.464615385	1.659230769	2.456153846	1.283076923	3.083846154	2.679230769	3.18	3.962307692	1.769230769
29	Bench Press 28	10	4	2.114328387	FALSE	0.761845267	1.21	1.05875	2.20125	1.32875	1.99875	1.1225	3.38375	2.33625	2.95375	3.315	0.75
30	Bench Press 29	7	10	1.032574792	FALSE	0.976022429	0.9	0.71	5	0.93	3.19	1.75	4.9	3.91	3.44	5	2
31	Bench Press 30	5	3	0.815296774	FALSE	3.571051642	1.46	0.75	5	1.56	2.07	1.93	3.32	2.82	2.42	2.82	1
32	Bench Press 31	6	4	0.349872727	FALSE	0.327806154	2.04	1.22	5	2.46	3.94	1.67	2.76	5	5	5	3
33	Bench Press 32	6	10	0.483162888	FALSE	0.804747893	2.15	0.95	2.6	2.05	2.86	1.23	5	5	3.65	4.67	2
34	Bench Press 33	6	3	1.198764706	FALSE	2.354965094	2.06	0.58	2.21	1.07	0.23	1.35	1.18	0.47	3.21	0.93	0
35	Bench Press 34	5	5	0.432179567	FALSE	3.498512344	0.9	0.54	1.796666667	0.966666667	1.013333333	0.95	2.306666667	1.473333333	1.666666667	1.746666667	0.666666667
36	Bench Press 35	5	3	0.366666667	FALSE	0.522490343	0.9	0.54	1.796666667	0.966666667	1.013333333	0.95	2.306666667	1.473333333	1.666666667	1.746666667	0.666666667
37	Bench Press 36	8	5	0.998547032	FALSE	0.727320227	1.506315789	1.110526316	2.222105263	0.991578947	1.576315789	0.967368421	3.828947368	2.327368421	4.057368421	2.947894737	0.842105263

Figure 2: Master Excel File

Feature engineering is then done on this master excel file (Figure 2) to generate specific deltas or ratios that would best assist the machine learning model. This is done in the same Python script as the model training. Doing the feature engineering here allowed the team to modify the engineered features without running the feature extractors along side it. The block of code in Figure 3 is what runs before the model training begins and is how the features are engineered.

```python
# Feature engineering (same as Multi-Modal)
au_cols = [col for col in numeric_cols if col.startswith('AU')]
if len(au_cols) > 0:
    df['AU_sum'] = df[au_cols].sum(axis=1)
    df['AU_mean'] = df[au_cols].mean(axis=1)
    df['AU_max'] = df[au_cols].max(axis=1)
    df['AU_std'] = df[au_cols].std(axis=1)

    if 'AU04_max' in au_cols and 'AU07_max' in au_cols:
        df['AU04_07_interaction'] = df['AU04_max'] * df['AU07_max']
    if 'AU06_max' in au_cols and 'AU12_max' in au_cols:
        df['AU06_12_interaction'] = df['AU06_max'] * df['AU12_max']
    if 'AU09_max' in au_cols and 'AU10_max' in au_cols:
        df['AU09_10_interaction'] = df['AU09_max'] * df['AU10_max']
    if 'AU25_max' in au_cols and 'AU26_max' in au_cols:
        df['AU25_26_interaction'] = df['AU25_max'] * df['AU26_max']
    if 'AU04_max' in au_cols and 'AU06_max' in au_cols:
        df['AU04_06_interaction'] = df['AU04_max'] * df['AU06_max']

if 'detection_rate' in numeric_cols and 'confidence_mean' in numeric_cols:
    df['quality_score'] = df['detection_rate'] * df['confidence_mean']

if 'confidence_mean' in numeric_cols and 'confidence_std' in numeric_cols:
    df['confidence_cv'] = df['confidence_std'] / (df['confidence_mean'] + 1e-6)

if 'd_value' in numeric_cols:
    if 'AU_sum' in df.columns:
        df['body_face_strain'] = df['d_value'] * df['AU_sum']
    if 'AU_mean' in df.columns:
        df['d_value_x_AU_mean'] = df['d_value'] * df['AU_mean']
    if 'au_highly_active_count' in numeric_cols:
        df['d_value_x_active_AUs'] = df['d_value'] * df['au_highly_active_count']

if 'rep_change_s' in numeric_cols and 'rep_count' in numeric_cols:
    df['avg_rep_duration'] = df['rep_change_s'] / (df['rep_count'] + 1)
    if 'AU_sum' in df.columns:
        df['tempo_x_facial_strain'] = df['avg_rep_duration'] * df['AU_sum']
    if 'd_value' in numeric_cols:
        df['tempo_x_body_movement'] = df['avg_rep_duration'] * df['d_value']

if 'rep_count' in numeric_cols:
    df['rep_count_sq'] = df['rep_count'] ** 2
    if 'au_highly_active_count' in numeric_cols:
        df['strain_per_rep'] = df['au_highly_active_count'] / (df['rep_count'] + 1)
    if 'AU_sum' in df.columns:
        df['AU_sum_per_rep'] = df['AU_sum'] / (df['rep_count'] + 1)
    if 'd_value' in numeric_cols:
        df['d_value_per_rep'] = df['d_value'] / (df['rep_count'] + 1)
    df['lift_type_x_reps'] = df['Lift_Type_Code'] * df['rep_count']
```

Figure 3: Feature Engineering Code

Once the final set of features are ready, they are fed into a Light Gradient Boosting Machine (LGBM) + eXtreme Gradient Boosting (XGBoost) ensemble model. Combining the outputs via ensembling produces a more robust model providing higher estimated accuracy than either model alone given the limited dataset size. Once training is completed, the model weights are saved as .bin and .joblib files which can be used to estimate the RPE of videos the model has never seen before. This is done in the UI pipeline, which follows the exact same format but instead of taking a dataset as input, it takes a single new video of the users lifting. This video is passed through the pipeline and the trained model outputs an estimated RPE.

2.3 Computer Vision Feature Extraction

Each computer vision module aims to take a preprocessed video, extract a specific physiological metric, and condense the extracted data into a set of values to be used as input to the LGBM model. The three metrics being analyzed are facial strain, bar speed, and form, each of which has been shown to correlate with RPE[16][18].

2.3.1 Facial Strain Extraction

The primary functional requirement of this module is to extract Facial Action Units (AUs) from exercise videos to estimate RPE. The module must support robust face detection to isolate the weightlifter despite noisy gym environments, variable lighting, equipment obstructions, and multiple camera angles. Processed features are filtered for key exertion indicators (e.g., brow furrowing, AU04 [14]), retaining only their maximum recorded values for machine learning. The module also generates visualized overlays for debugging and features API compatibility.

To satisfy the functional requirements, the facial extraction architecture is divided into three modular systems.

1. **Preprocessor:** Subsystem that scans every frame of the video, identifies the primary weightlifter, and generates a stabilized cropped video of the individual's head region.
2. **Feature Extraction Engine:** The core binary interface that processes the cropped video and extracts frame-by-frame raw facial strain data (Action Unit Intensities).
3. **Data Cleaning and Imputation:** Subsystem designed to isolate key columns or variables that relate most to physical exertion, identify maximum achieved values of

the decided variables for the LGBM model, and repairs failed video extractions through averaged imputations.

During development, several environmental and systemic constraints resulted in key design decisions:

- **Background Noise & Dynamic Movement:** Early testing revealed OpenFace struggled to maintain a consistent facial lock due to background noise and dynamic lifting movements. We addressed this by integrating MediaPipe [4] as a preprocessing step to efficiently identify the largest person and crop the video to their head.
- **Camera Jitter:** Updating the MediaPipe bounding box frame-by-frame introduced artificial camera jitter that worsened feature extraction. We resolved this by using a fixed crop based on the median center of the lifter's trajectory across the entire video.
- **Data Overload:** To manage the sheer volume of irrelevant data generated per video, the system filters the raw output to retain only the maximum observed values of essential variables. This reduces the amount of processing required and isolates the peak strain moments.
- **Small Dataset & Obscurity:** Ignoring videos that failed face detection (due to extreme angles or obscurity) would severely limit our already small dataset. We developed an imputation strategy where failed videos have their missing features filled using the mean AU values of successful videos sharing the exact same exercise type and user-reported RPE.

- **Memory Leaks:** Processing hundreds of high-resolution videos caused memory leakages. We implemented a chunk batch-processing design, limiting the system to process 30 videos at a time before intentionally exiting and freeing memory space.

2.3.2 Bar Speed Tracking

The bar speed tracking was a core subsystem to quantify fatigue and is a primary objective indicator of an athlete's Rate of Perceived Exertion (RPE). The requirements dictated that the system must process standard 1080p, 60 fps RGB video inputs to track the vertical displacement of the barbell during three primary compound movements, squat, bench press, and deadlift.

During the initial design phase, the pipeline utilized MMDetection, a bounding-box object detection framework, to track the barbell. However, after a strenuous testing process MMDetection was found to be both computationally heavy and prone to identifier instability. The system frequently lost the bounding box when the barbell passed in front of dark clothing, power rack safety pins, or other gym equipment, leading to crucial data loss and unrealistic velocity curves.

To satisfy the project's reliability constraint (maintaining tracking in $\geq 95\%$ of frames), the design was changed to a hybrid computer vision approach. Instead of continuous object detection, the bar speed tracking was redesigned to initialize a tracking point using MMPose, an open-source pose estimation toolbox. MMPose locates the lifter's wrist or the barbell sleeve in the initial frame. Once this coordinate is locked, the pipeline hands the tracking over to OpenCV, which utilizes its lightweight optical flow algorithms to track the movement vector frame-by-frame.

A critical design challenge within this pipeline was translating the raw pixel displacement into standard physical units (meters per second). To achieve this without requiring users to input camera distances, the pipeline was designed to use the universally standard 45cm diameter of an Olympic weight plate as a fixed reference. Finally, the bar speed tracking subsystem includes a custom algorithmic filter designed to automatically crop static pauses and segment the remaining motion data into distinct concentric (lifting) and eccentric (lowering) phases. This design allows the pipeline to isolate the specific velocity loss (Δv) between the first and final repetitions, which is ultimately fed as given to the XGBoost/LightGBM ensemble model.

2.3.3 Form Detection

The form detection computer vision module aims to analyze the change in form from the start of the set to the end of the set. It takes a preprocessed video as input, and outputs a singular metric representing the change in form between the first and last rep of the set. A large difference in form between the first and last rep represents a significant breakdown in lifting form, which can be a useful predictor of RPE for that set.

The MMPose computer vision model was selected to fulfil the requirements of the form detection module. It works by placing "key points" on the major joints of a person's body in each frame of an input video. The model outputs a set of key point pixel coordinates for each frame, resulting in a large data stream for an entire video. This data stream then must be parsed to identify the start and finish of each rep to obtain the first and last rep of the set. A difference function is then applied to those two reps to condense the stream into a single metric representing the change in form between the start to the end of the set. For each lift,

only specific joint positions are relevant depending on what muscles are used for that exercise, and therefore the difference function needs to be applied only to the lift-relevant joints.

Initial implementations of the MMPose model had poor key point tracking accuracy, largely due to obstructions in input videos blocking parts of the body from view in certain frames. To address this limitation of the MMPose model, rules on key point placement were added to enforce consideration of inter-frame motion and expected body positions for each lift. This allows for more intelligent body position extraction than the MMPose default frame-by-frame approach.

2.4 Model Design

RPEasy uses a machine learning regressor model responsible for predicting the Rate of Perceived Exertion (RPE). A regression model is a machine learning tool that estimates the relationship between a dependent variable (target) and an independent variable (predictors) which has a continuous outcome. Due to the constrained data size of 750+ rows and the structure of the extracted features, a deep neural network was out of the scope of this project. To prevent overfitting on a small dataset, the team choose a gradient-boosting ensemble architecture for the model.

This consists of a weighted ensemble combination of a Light Gradient Boosting Machine (LightGBM) and eXtreme Gradient Boosting (XGBoost). This combination was chosen as both are decision tree regressor models, but they both grow their trees differently. LightGBM uses leaf-wise growth, while XGBoost uses level-wise growth. The difference in tree growth allows their individual prediction errors to be less correlated. Ensembling them

together yields a more generalized prediction than just using the models independently. When testing different weighted splits during the design phase, a split of 60% LightGBM and 40% XGBoost was determined to be optimal for this project.

To prevent any further overfitting, the hyperparameters of the model had to be tuned with this goal in mind. Both models were configured to have a very low learning rate (e.g. 0.01) paired with a high number of estimators (2600). Additionally, the models used aggressive L1 and L2 regularization and feature sub-sampling. These hyperparameters help the model learn conservative, generalized patterns instead of memorizing the specific training data.

To test the model's accuracy and performance, Mean Absolute Error (MAE) was selected as the primary loss function and evaluation metric. MAE was chosen over Root Mean Square Error because it scales linearly, which aligns better with measuring the error in RPE estimation. The MAE was used to compare the model's performance with the project's target specification of ±0.5 RPE.

Additionally, because RPE is a continuous variable, but the dataset was skewed towards high-intensity lifts, the model used 7-fold Stratified Cross-Validation in its design. This overcame the skewness by binning the target RPEs into distinct categories (low, medium, and high) so that each validation fold would contain a proportionally representative sample of all intensity levels.

2.5 Application UI

The User Interface (UI) was designed to act as the bridge between the computationally heavy machine learning backend and the athlete in a dynamic gym environment. The

primary design constraint for the UI was operational accessibility, it needed to be a responsive web application that allowed for a smooth video upload process.

Functionally, the UI pipeline begins when the user submits a video file. To prevent frontend freezing and ensure a smooth user experience, the system was designed with an asynchronous processing architecture. Upon upload, the backend routes the media payload through the three parallel feature extraction modules, OpenCV (bar speed), MMPose (form breakdown and path deviation), and OpenFace (facial action units and strain analysis).

Once these three models finish processing, the UI pipeline aggregates the isolated features, such as the change in bar speed, form breakdown, and facial strain, and passes them into the trained ensemble regressor. A key specification in the UI design was transparency. The UI was designed to generate a transparent breakdown of the contributing physiological factors. By mapping the highest-weighted features from the LightGBM model to specific text prompts, the UI displays dynamic feedback, such as "Significant Velocity Loss Detected" or "High Facial Strain", giving the user objective insights to adjust their training volume and mitigate the risk of overtraining.

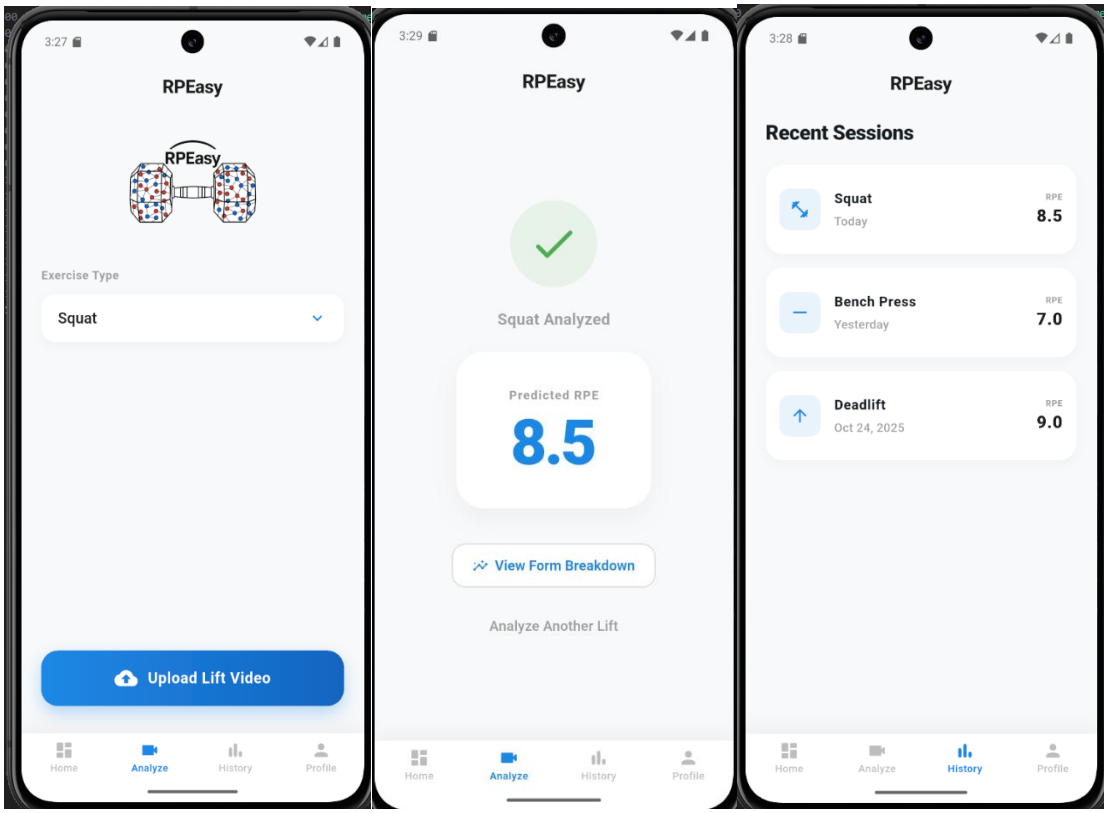


Figure 4: RPEasy UI

3. Implementation

3.1 Dataset Creation

The physical compilation and processing of the RPEasy dataset was a highly manual and time-consuming phase of the project. Additionally, to get all of the videos ready for feature extraction via OpenFace, OpenCV, and MMPose, the dataset required lots of manual preprocessing to achieve the best results.

The dataset creation began with manually downloading publicly available weightlifting videos along with the team recording 10% of the videos themselves. Each video had to be reviewed for usability from beginning to end before including it into the dataset. When

reviewing the videos, the feature extractors had to be kept in mind as videos with extreme background noise, or poor camera angles that hid barbell tracking were discarded from the dataset.

To ensure consistency when feeding the dataset videos into the feature extractors, all of the videos must be converted into an MP4 video format. Varying sources from different devices such as iPhones caused some of the video formats to be MOV files. Due to the way the feature extractors were designed, they were unable to correctly extract features from MOV files, hence they had to be converted to MP4 files.

```
def ensure_mp4(input_path):  1 usage  lukehunker
    """
    If input_path is not .mp4, convert to .mp4 using ffmpeg and return new path.
    If already .mp4, return input_path.
    """
    ext = os.path.splitext(input_path)[1].lower()
    if ext == '.mp4':
        return input_path
    mp4_path = os.path.splitext(input_path)[0] + '_converted.mp4'
    if not os.path.exists(mp4_path):
        print(f"[INFO] Converting {input_path} to MP4: {mp4_path}")
        cmd = [
            'ffmpeg', '-y', '-i', input_path,
            '-c:v', 'libx264', '-preset', 'fast', '-crf', '22',
            '-c:a', 'aac', '-movflags', '+faststart',
            mp4_path
        ]
        try:
            subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
        except Exception as e:
            print(f"[ERROR] ffmpeg conversion failed: {e}")
            raise
    return mp4_path
```

Figure 5: Convert File Type to MP4 Function

Figure 5 contains the function that was ran on the videos in the dataset that would automatically convert the video format and encoding to type MP4 if it detected any other formatting. This code was run before feeding the video into the feature extractors, saving the team from manually converting all of the videos to the correct type. Additionally, to complete the formatting and standardization of the dataset, each video was manually renamed to a systematic naming convention (e.g. "Squat 75") to keep the data organized.

The most labor-intensive phase of the dataset creation involved the precise editing of the video files. Each video had to be manually cropped to start at the exact frame the lifter initiated the movement and end at the exact frame the set was completed. This removed any noise that was introduced by the un-racking, and re-racking, such as the bar speed being tracked during those phases of the lift. Additionally, for the high-quality videos that met the criteria for data augmentation, the team manually had duplicate them and manually crop out the final repetitions from the video. This allowed the team to engineer new, valid data points that represented lower RPE efforts without needing to source new footage.

The final step to completing the dataset was getting all of the footage labeled to establish ground truth RPE values. RPE is a complex metric to evaluate, so the team contracted Trevor Felix, a certified level 5 personal trainer at GoodLife Fitness. This required a significant amount of time and project budget, as Trevor had to watch every cropped video from start to finish and assign an accurate, unbiased RPE label to each file. After each video was labeled with a ground truth label, the dataset was now considered complete and ready for the extraction of facial strain, bar speed, and form features.

3.2 Facial Strain Extraction- OpenFace

The facial strain analysis unit was developed using Python 3.11 [1], MediaPipe [4] for spatial tracking, and the compiled C++ OpenFace toolkit [6]. OpenFace is a state-of-the-art, open-source facial behavior analysis toolkit capable of facial landmark detection, head pose estimation, and Facial Action Unit (AU) recognition [6]. For the scope of this project, OpenFace will be used strictly as a feature extraction engine to measure the intensity of specific Facial Action Units associated with physical exertion. The implementation is

divided into the three key stages mentioned in the design section: the preprocessor, the OpenFace extraction engine, and the data cleaning and aggregation pipeline.

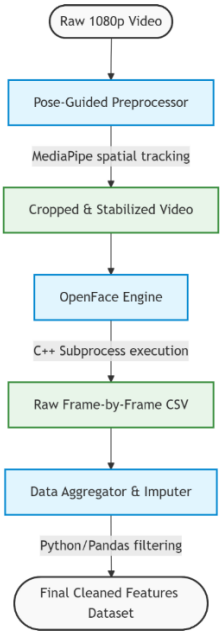


Figure 6: OpenFace Processing Pipeline

3.2.1 Pose-Guided Preprocessor Implementation

The preprocessing phase is handled by the PoseGuidedFaceDetector class, tasked with isolating the primary lifter and removing jitter to allow a stable consistent video of the face for OpenFace to take as input.

The class utilizes the Medipipe library to scan the full 1080p video frame and locate the user’s skeletal keypoints [4]. To eliminate the algorithmic tracking jitter the bounding box is not updated frame-by-frame. Instead, the script performs an initial pass of the entire video to calculate the median position of the lifter's head across all frames, establishing a fixed, stabilized crop region. The second pass then crops every frame to this exact coordinate region.

```python
        # Get median center and max dimensions
        x1_median = int(np.median([b[0] for b in valid_bboxes]))
        y1_median = int(np.median([b[1] for b in valid_bboxes]))
        x2_median = int(np.median([b[2] for b in valid_bboxes]))
        y2_median = int(np.median([b[3] for b in valid_bboxes]))

        # Use max dimensions to ensure face fits in all frames
        max_width = max(b[2] - b[0] for b in valid_bboxes)
        max_height = max(b[3] - b[1] for b in valid_bboxes)

        # Calculate fixed crop region
        center_x = (x1_median + x2_median) // 2
        center_y = (y1_median + y2_median) // 2

        crop_width = int(max_width * 1.2)
        crop_height = int(max_height * 1.2)
```

Figure 7: Poseguided Face Detector Code for Obtaining Crop Region Via MediaPipe

3.2.2 OpenFace Extraction

The core feature extraction engine uses a python wrapper to execute compiled OpenFace binaries as a subprocess [6]. The run_OpenFace() function directs the compiled binary from the toolkit to process the already stabilized, cropped videos. With the goal of optimizing execution times, specific command line flags were passed to disable unused modules in OpenFace such as the 3D landmarks and gaze tracking modules. This forced the binary to focus primarily on the extraction of facial action units, saving costs on time and compute.

```python
cmd = [
    OPENFACE_BIN,
    "-f", video_path,
    "-out_dir", OUTPUT_DIR,
    "-aus",  # Only extract Action Units (AUs)
    "-multi_view", "1",
]

result = subprocess.run(cmd, check=True, capture_output=True, text=True)
```

Figure 8: Code Block for Running OpenFace C++ Binary Via Subprocess

3.2.3 Imputation and Data Cleaning Implementation

Once the OpenFace CSV was generated, the OpenFaceExtractor class manages the data aggregation. To ensure resting faces are not misinterpreted as straining, the system filters for frames where OpenFace reported a successful face detection. With maximum strain being the most critical indicator of the person's RPE, the _extract_au_features() method was

developed using NumPy [7] to iterate through specific EXERTION_AUS (e.g., AU04 for brow lowering, AU26 for jaw dropping) and isolate the maximum observed intensity for the final dataset:

```
def _extract_au_features(self, df):

    features = {}

    for au in self.EXERTION_AUS:
        if au not in df.columns:
            continue

        au_data = df[au].values
        prefix = au.replace('_r', '')

        if self.max_only:
            # Extract only max features for RPE prediction
            features[f'{prefix}_max'] = np.max(au_data)
```

Figure 9: Feature Extracting Isolating for Maximum Exertion AU Values

After isolating specific columns and their maxes, a data recovery script was implemented for videos where extreme recording conditions caused OpenFace to fail completely at detecting and analyzing the lifters face. Using the Pandas library [8], the system identifies the failed video's reported RPE and exercise type. It then locates all successfully processed videos matching those exact parameters and imputes the missing data by calculating the mathematical mean of those peer features, ensuring no data points are entirely lost:

```
# Find peer videos matching the exact exercise type and RPE
similar_videos_mask = (
    (rpe_df['exercise_type'].str.lower() == video_exercise) &
    (rpe_df['rpe'] == video_rpe) &
    (~rpe_df['video_name'].isin(missing_videos))
)
similar_video_names = rpe_df[similar_videos_mask]['video_name'].tolist()
similar_features = features_df[features_df['video_name'].isin(similar_video_names)]
imputation_method = f"{video_exercise} + RPE={video_rpe}"

# Calculate the mean of features from similar videos for imputation
mean_features = similar_features[numeric_cols].mean()
```

Figure 10: Code Block for Aggregating Missing Data Points

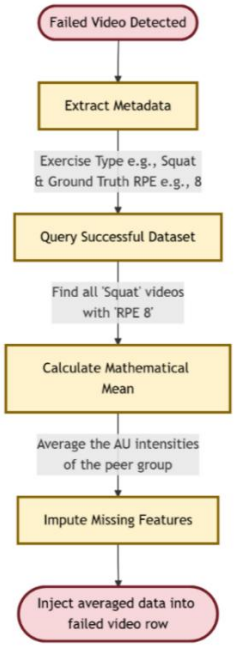


Figure 11: Data Aggregation Pipeline

3.3 Bar Speed Tracking- OpenCV

The implementation of the bar speed tracking module was achieved through a multi-stage Python pipeline utilizing the OpenCV and MMPose libraries. The primary objective of this module was to process raw RGB video (1080p, 60 fps) and output a standardized CSV array of velocity features, specifically the velocity loss (Δv) between the initial and final repetitions, to be used by the LGBM and XGBoost ensemble model.

3.3.1 Initialization and Point Tracking

The execution begins with a calibration script that reads the initial frame of the uploaded video. To locate the barbell, the script invokes the MMPose API to perform 2D human pose estimation, this targets the lifter's wrist as the point to track as it is reliable proxy for the barbell's position. Once the precise (x, y) pixel coordinates of the wrist are isolated in the first frame, these coordinates are passed to OpenCV. OpenCV's Lucas-Kanade optical flow

algorithm (*cv2.calcOpticalFlowPyrLK*) was implemented to track this specific cluster of pixels across all subsequent frames. This method proved highly efficient, consistently maintaining the tracking point while avoiding the heavy processing overhead associated with frame-by-frame object detection.

3.3.2 Dimensional Scaling and Calibration

A critical implementation challenge was converting the raw pixel displacement tracked by OpenCV into real-world physical units (meters per second) required for accurate RPE estimation. To implement this functionality, an automated scaling function was written. The algorithm detects the bounding box of the weight plates visible on the barbell. By mapping the pixel height of this bounding box to the universally standard 45-centimeter diameter of an Olympic weight plate, the script calculates a dynamic pixel-to-meter conversion ratio. This ratio is applied to the frame-by-frame vertical displacement data, ensuring that athletes standing at different distances from the camera generate uniformly scaled velocity data.

3.3.3 Data Processing and Phase Segmentation

The raw position data output by the optical flow algorithm naturally contains high-frequency noise due to minor camera shakes or micro-adjustments by the lifter. To clean this data, a Butterworth low-pass filter was implemented using the SciPy library, smoothing the vertical displacement curve. Following noise reduction, a custom peak-detection algorithm was implemented using *scipy.signal.find_peaks* to identify the inflection points of the lift. By analyzing the local minima and maxima of the position curve, the script automatically crops out periods of static rest (such as the lifter bracing before the first

repetition) and segments the active motion into concentric (upward) and eccentric (downward) phases.

3.3.4 Feature Extraction for the Machine Learning Model

Once the distinct repetitions are isolated, the final stage of the implementation calculates the peak and average velocities for each concentric phase. The script stores the velocity of the first repetition and compares it against the velocity of the final repetition to calculate the overall velocity drop-off (Δv). This specific fatigue metric, alongside the raw repetition count and average concentric duration, is packaged into an array and appended to the dataset, ready to be merged with the facial strain and form breakdown features for the final XGBoost/LightGBM estimation.

3.4 Form Detection - MMPose

Form detection feature extraction was accomplished with MMPose, an open-source python toolkit for body pose detection in images and videos which places key points on major joints of the body. It was implemented using logical rules that limit potential key point placements to decrease tracking errors and ensure usable data. These rules can be broken down into two categories: rules for expected body movement between frames which can be applied to all videos, and rules for expected body positions for specific exercises. After key points are extracted and written to a json file, a python script is used to identify individual reps within the set and calculate the difference in form between the first and last rep of the set, outputting a “difference metric” that is useable for input to the LGBM/XGBoost model.

3.4.1 General Motion Rules

The most important key point motion rule enforced for all lifts is a jump clamping rule. This rule aims to reduce teleportation errors, where a key point suddenly jumps a large distance from one frame to the next. Before jump clamping was implemented, teleportation errors would sometimes occur when a specific joint was not visible in a given frame, such as when view of a joint was obstructed by a squat rack or part of the weights, resulting unpredictable key point placement. Clamping was implemented with a function which will reject large jumps in key point position between two frames, keeping the key point's previous position instead. To avoid "freezes", where a key point is clamped in the same position for a long time, a limit of 4 consecutive clamped frames is allowed before a large position jump is accepted, attempting to reestablish joint tracking.

```
def clamp_jumps(prev, cur):
    # Reject crazy jumps. Returns (clamped_output, was_clamped_per_joint).
    clamped = np.zeros(cur.shape[0], dtype=bool)
    if prev is None:
        return cur, clamped
    sw = shoulder_width(prev)
    if sw <= 1e-3:
        return cur, clamped
    max_jump = MAX_JUMP_REL * sw
    out = cur.copy()
    for j in range(cur.shape[0]):
        if not (np.all(np.isfinite(prev[j])) and np.all(np.isfinite(cur[j]))):
            continue
        d = cur[j] - prev[j]
        if float(d[0]*d[0] + d[1]*d[1]) > max_jump*max_jump:
            out[j] = prev[j]
            clamped[j] = True
    return out, clamped
```

Figure 12: Jump Clamping Function for Squats

A simple motion-smoothing rule is also applied to all key points, using an exponential moving average to create consistent key point paths while avoiding sudden changes in position and velocity between frames.

```python
def ema(prev, cur, alpha=EMA_ALPHA):
    if prev is None:
        return cur
    out = cur.copy()
    for j in range(cur.shape[0]):
        cx, cy = cur[j]
        px, py = prev[j]
        c_ok = np.isfinite(cx) and np.isfinite(cy)
        p_ok = np.isfinite(px) and np.isfinite(py)
        if c_ok and p_ok:
            out[j, 0] = alpha*cx + (1-alpha)*px
            out[j, 1] = alpha*cy + (1-alpha)*py
        elif (not c_ok) and p_ok:
            out[j] = prev[j]
    return out
```

Figure 13: Motion Smoothing Function

3.4.2 Exercise-Specific Rules

Each of the three exercises used in this project have different challenges in ensuring proper tracking, and therefore different key point placement rules to address these challenges. Many of these differences can be addressed by analyzing only the lift-relevant joints for each exercise, or by tuning parameters like the maximum allowable key point jump between frames. The bench press exercise proved to be the most challenging lift to achieve accurate tracking with, and as such it required additional logical rules.

Bench press has some important differences from the other two lifts: it is an upper-body movement performed lying down and is often recorded from a profile view, where only some joints may be visible. These differences initially resulted in low tracking accuracy, requiring unique rules to produce usable data. Three additional rules were created, which each rely on an assumption about body positions during the exercise. Firstly, separation between the arms is enforced to address the issue of MMPose sometimes placing both sets of arm key points on the same arm. Next, elbow key point position is controlled using the assumption that elbow joints should remain relatively level with each other throughout the

lift. Finally, forearm parallelism is enforced under the assumption that the angle of the forearms remains relatively vertical and parallel throughout the movement.

```python
def enforce_arm_separation(prev, cur):
    if prev is None:
        return cur
    sw = shoulder_width(prev)
    thr = max(MIN_ARM_SEP_PX, MIN_ARM_SEP_REL * sw) if sw > 1e-3 else MIN_ARM_SEP_PX
    thr2 = thr * thr
    out = cur.copy()
    pairs = [(L_ELBOW, R_ELBOW), (L_WRIST, R_WRIST)]
    for a, b in pairs:
        if not (np.all(np.isfinite(out[a])) and np.all(np.isfinite(out[b]))):
            continue
        d = out[a] - out[b]
        if float(d[0]*d[0] + d[1]*d[1]) >= thr2:
            continue
        if np.all(np.isfinite(prev[a])) and np.all(np.isfinite(prev[b])):
            da = out[a] - prev[a]
            db = out[b] - prev[b]
            ma = float(da[0]*da[0] + da[1]*da[1])
            mb = float(db[0]*db[0] + db[1]*db[1])
            if ma > mb:
                out[a] = prev[a]
            else:
                out[b] = prev[b]
        else:
            if np.all(np.isfinite(prev[a])):
                out[a] = prev[a]
            if np.all(np.isfinite(prev[b])):
                out[b] = prev[b]
    return out
```

Figure 14: Arm Separation Function

```python
def enforce_elbow_position(prev, cur, kpt_scores=None):
    if prev is None:
        return cur
    sw = shoulder_width(cur)
    if sw <= 1e-3:
        sw = shoulder_width(prev)
    if sw <= 1e-3:
        return cur
    max_dy = ELBOW_PARALLEL_MAX_Y * sw
    out = cur.copy()
    pairs = [(L_ELBOW, R_ELBOW), (L_WRIST, R_WRIST)]
    for a, b in pairs:
        if not (np.all(np.isfinite(out[a])) and np.all(np.isfinite(out[b]))):
            continue
        dy = abs(float(out[a, 1] - out[b, 1]))
        if dy <= max_dy:
            continue
        conf_a = 0.5
        conf_b = 0.5
        if kpt_scores is not None:
            conf_a = float(kpt_scores[a])
            conf_b = float(kpt_scores[b])
        if conf_a >= conf_b:
            out[b, 1] = out[a, 1]
        else:
            out[a, 1] = out[b, 1]
    return out
```

Figure 15: Elbow Level Function

```python
def enforce_forearm_parallelism(prev, cur, kpt_scores=None):
    if prev is None:
        return cur
    out = cur.copy()
    le_ok = np.all(np.isfinite(out[L_ELBOW])) and np.all(np.isfinite(out[L_WRIST]))
    re_ok = np.all(np.isfinite(out[R_ELBOW])) and np.all(np.isfinite(out[R_WRIST]))
    if not (le_ok and re_ok):
        return out
    l_vec = out[L_WRIST] - out[L_ELBOW]
    r_vec = out[R_WRIST] - out[R_ELBOW]
    l_len = float(np.sqrt(l_vec[0]**2 + l_vec[1]**2))
    r_len = float(np.sqrt(r_vec[0]**2 + r_vec[1]**2))
    if l_len < 1e-3 or r_len < 1e-3:
        return out
    l_angle = float(np.arctan2(l_vec[1], l_vec[0]))
    r_angle = float(np.arctan2(r_vec[1], r_vec[0]))
    angle_diff = abs(l_angle - r_angle)
    if angle_diff > np.pi:
        angle_diff = 2 * np.pi - angle_diff
    max_angle = np.radians(FOREARM_MAX_ANGLE_DEG)
    if angle_diff <= max_angle:
        return out
    l_conf = 0.5
    r_conf = 0.5
    if kpt_scores is not None:
        l_conf = (float(kpt_scores[L_ELBOW]) + float(kpt_scores[L_WRIST])) / 2.0
        r_conf = (float(kpt_scores[R_ELBOW]) + float(kpt_scores[R_WRIST])) / 2.0
    if l_conf >= r_conf:
        out[R_WRIST, 0] = out[R_ELBOW, 0] + r_len * float(np.cos(l_angle))
        out[R_WRIST, 1] = out[R_ELBOW, 1] + r_len * float(np.sin(l_angle))
    else:
        out[L_WRIST, 0] = out[L_ELBOW, 0] + l_len * float(np.cos(r_angle))
        out[L_WRIST, 1] = out[L_ELBOW, 1] + l_len * float(np.sin(r_angle))
    return out
```

Figure 16: Forearm Parallelism Function

3.4.3 Rep Detection and Difference Metric Calculation

Once a stream of key point coordinates is generated with MMPose, a python script is used to generate a singular "difference metric" representing change in form between the first and last rep of a set. To accomplish this, the script first identifies each rep by analyzing the vertical position of a relevant joint (shoulders for squat, wrist for bench press and deadlift). Next, relevant joint positions are compared between the first and last rep of the set using a squared difference function, before scaling the difference output by the key point placement confidence. A different set of joints is used for this calculation depending on the exercise, according to the biomechanics of the lift: hips, knees and ankles for squat, shoulders, elbows and wrists for bench press, and the full body for deadlift. The final difference metric is then written to the master results excel file for LGBM/XGBoost training or inference.

```
# Weighted D: each joint's squared diff is scaled by its confidence
diff_sq = np.sum((E - L) ** 2, axis=2)
weighted = diff_sq * joint_weights[None, :]
D = float(np.sqrt(np.sum(weighted) / np.sum(joint_weights) / N_SAMPLES))
```

Figure 17: Confidence-Weighted Difference Metric Function

3.5 Model Training

The model training was implemented using a Python script (LGBMMod2_Ensemble.py) utilizing the scikit-learn, lightgbm, and xgboost libraries as seen in Figure 18. The script takes the master feature spreadsheet as input, executes the cross-validation training loop, and saves the final trained model for the application UI.

```
import pandas as pd
import numpy as np
import lightgbm as lgb
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import warnings
from datetime import datetime
```

Figure 18: LGBMMod2_Ensemble.py Libraries

The specific hyperparameter dictionaries for the LGBMRegressor and the XGBRegressor can be seen in Figure 19. Both models were initialized with a very small learning rate and paired with a large number of estimators (2600) to ensure slow, stable convergence. To further prevent the models from memorizing the limited dataset, L1 and L2 regularization limits (reg_alpha = 0.13, reg_lambda = 0.38) were hardcoded into the configuration. Furthermore, stochastic elements were used such as restricting the models to only use 74% of the data rows (subsample = 0.74) and 64% of the features (colsample_bytree = 0.64) for each boosting iteration.

```python
# LGBM hyperparameters
lgbm_params = {
    'objective': 'regression',
    'metric': 'mae',
    'boosting_type': 'gbdt',
    'n_estimators': 2600,
    'learning_rate': 0.0095,
    'num_leaves': 33,
    'max_depth': 9,
    'min_child_samples': 4,
    'min_child_weight': 0.036,
    'subsample': 0.74,
    'subsample_freq': 6,
    'colsample_bytree': 0.64,
    'reg_alpha': 0.13,
    'reg_lambda': 0.38,
    'min_gain_to_split': 0.0005,
    'max_bin': 216,
    'verbose': -1,
    'n_jobs': -1
}

# XGBoost hyperparameters
xgb_params = {
    'objective': 'reg:absoluteerror',
    'n_estimators': 2600,
    'learning_rate': 0.01,
    'max_depth': 9,
    'min_child_weight': 4,
    'subsample': 0.74,
    'colsample_bytree': 0.64,
    'reg_alpha': 0.13,
    'reg_lambda': 0.38,
    'tree_method': 'hist',
    'n_jobs': -1
}
```

Figure 19: Model Hyperparameters

Once the master feature list is loaded, and the irrelevant features are dropped. The data is then prepped for cross-validation by converting the RPE target variables into discrete categorical bins using pandas. StandardScaler() is fitted to the training fold and applied to the validation fold. This standardization ensures that features with different scales do not disproportionately bias the gradient boosting weight updates.

Once the data is prepped and scaled, the script implements StratifiedKFold with 7 splits training both the LGBMRegressor and the XGBRegressor as seen in Figure 20. Once both models are fitted to the training fold, they predict the RPE values for the validation fold. The ensemble implementation calculates the final prediction by taking a weighted average (0.6 * lgbm_preds) + (0.4 * xgb_preds). numpy.clip is used to enforce a hard boundaries from 0-10 ensuring the ensemble never outputs mathematically impossible RPE values.

```python
skf = StratifiedKFold(n_splits=7, shuffle=True)
all_y_true = []
all_lgbm_preds = []
all_xgb_preds = []
all_ensemble_preds = []

print("\nTraining models on CV folds...")
for fold_idx, (train_idx, val_idx) in enumerate(skf.split(X, y_bins), 1):
    print(f"  Fold {fold_idx}/7...", end=" ")

    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    # Train LightGBM
    lgbm_model = lgb.LGBMRegressor(**lgbm_params)
    lgbm_model.fit(X_train_scaled, y_train)
    lgbm_preds = lgbm_model.predict(X_val_scaled)

    # Train XGBoost
    xgb_model = xgb.XGBRegressor(**xgb_params)
    xgb_model.fit(X_train_scaled, y_train)
    xgb_preds = xgb_model.predict(X_val_scaled)

    # Ensemble: weighted average
    ensemble_preds = 0.6 * lgbm_preds + 0.4 * xgb_preds
    ensemble_preds = np.clip(ensemble_preds, a_min: 1, a_max: 10)

    all_y_true.extend(y_val)
    all_lgbm_preds.extend(lgbm_preds)
    all_xgb_preds.extend(xgb_preds)
    all_ensemble_preds.extend(ensemble_preds)
```

Figure 20: LGBMMod2_Ensemble.py Training Loop

The finalized weights are exported using joblib and model-specific save functions (creating .txt, .json, and .pkl artifacts) as seen in Figure 21. These serialized files are then loaded directly into the RPEasy API backend to process live user uploads.

```python
# Save models
import joblib
import os
import json

output_dir = "../Train_Outputs"
os.makedirs(output_dir, exist_ok=True)

# Save LGBM model
lgbm_model_path = os.path.join(output_dir, f"lgbm_ensemble_model_{VERSION}.txt")
lgbm_final.booster_.save_model(lgbm_model_path)
print(f"\nSaved LightGBM model: {lgbm_model_path}")

# Save XGBoost model
xgb_model_path = os.path.join(output_dir, f"xgb_ensemble_model_{VERSION}.json")
xgb_final.save_model(xgb_model_path)
print(f"Saved XGBoost model: {xgb_model_path}")

# Save scaler
scaler_path = os.path.join(output_dir, f"ensemble_scaler_{VERSION}.pkl")
joblib.dump(scaler_final, scaler_path)
print(f"Saved scaler: {scaler_path}")
```

Figure 21: LGBMMod2_Ensemble.py Model Weight Saving

3.6 UI- RPEasy Application

The User Interface (UI) was implemented as a cross-platform mobile application utilizing the Flutter framework and the Dart programming language. The primary implementation objective was to create a frictionless, mobile-responsive bridge between the athlete in the gym and the computationally heavy machine learning backend. The architecture follows a modular, state-driven design centered around a main navigation container that routes the user between distinct operational tabs: Home, Analyze, History, and Profile.

3.6.1 Video Acquisition and Preprocessing Interface

The core functionality of the application resides within the Analyzer module (AnalyzerTab). To handle media input, the image_picker library was used, allowing users to select video files directly from their device's local gallery. Recognizing that raw gym footage often contains lengthy periods of static rest, a custom trimming dialog was implemented. This allows the user to input specific start and end timestamps, dynamically cropping the video before transmission. This implementation detail prevents the backend from wasting computational resources processing irrelevant frames. To maintain a smooth user experience during the transition from file selection to network transmission, an AnimatedSwitcher was utilized to cross-fade between the 'idle' state and a dynamic 'processing' loading screen.

3.6.2 Backend API Integration and State Management

Communication between the Flutter frontend and the estimation backend was implemented using a singleton class (RPEApiService). This service relies on the Dart http package to execute RESTful API calls. When an athlete uploads a lift, the system constructs a

MultipartRequest, bundling the .mp4 video alongside metadata such as the selected lift type (Squat, Bench Press, or Deadlift) and the optional crop timestamps. Due to the parallel execution of OpenFace, MMPose, and OpenCV, the backend is computationally intensive. The API service was engineered with a robust 10-minute timeout threshold and comprehensive try-catch error handling blocks to gracefully catch socket exceptions or network timeouts, ensuring the application does not crash during long processing windows.

3.6.3 Dynamic Data Rendering and Form Breakdown Insights

Once the backend completes its execution, it returns a serialized JSON packet containing the LightGBM/XGBoost RPE estimation and the isolated features. The UI implementation dynamically maps these values to the Document Object Model (DOM). The primary result screen displays the estimated RPE integer alongside the detected repetition count. To fulfill the design requirement of providing transparent feedback, an Insights screen was implemented to translate the raw machine learning arrays into easy-to-understand, color-coded health indicators.

For instance, the implementation uses threshold logic to parse the fatigue delta (Δv), a drop-off under 0.20 seconds registers as 'Stable Tempo' (green), while a drop-off exceeding 0.45 seconds triggers a 'High Fatigue' (red) alert. Similar threshold logic is applied to the MMPose D-metric to evaluate posture deviation, and OpenFace confidence scores to validate facial tracking reliability. The application integrates the video_player library to stream a secondary video file returned by the server, superimposing the OpenFace tracking overlay directly onto the athlete's original footage, providing visual proof of the feature extraction.

3.7 Final Project Budget

The project was allocated a maximum budget of $600 CAD. In the initial project blueprint, expenditures were estimated at approximately $380, primarily allocated for hardware accessories (such as lighting kits and external SSDs) and cloud GPU infrastructure. However, during the physical implementation phase, it was determined that the ambient lighting in standard commercial gyms was sufficient for the MMPose and OpenFace models to extract features accurately. Also, personal mobile devices and local computing resources successfully met the system's strict processing requirements. Consequently, all planned physical hardware and cloud infrastructure acquisitions were deemed unnecessary, reducing those specific category expenditures to $0.

The sole financial expenditure for the project was $480.25, which was allocated toward professional dataset labeling. While the original blueprint did not outline a budget for data annotation, our faculty supervisor recommended using an experienced third party to establish the ground truth for the machine learning models. Due to the Rate of Perceived Exertion (RPE) being a highly subjective metric, having the team self-annotate the 750+ video dataset would have introduced bias into the training pipeline. To mitigate this potential risk, the team hired a certified Level 5 personal trainer to objectively review and label the footage. Compensating the personal trainer allowed for an unbiased foundation that the LightGBM and XGBoost regressors could truly work on. Ultimately, this reallocation of resources kept the project well under the $600 limit and ensured success in the final product.

4. Testing

4.1 Facial Strain Extraction - OpenFace

The primary objective when evaluating the facial strain module was to verify OpenFace's ability to accurately process facial video data and extract key indicators of strain. The performance of this module was verified against the project's functional requirements, which required a face detection and strain analysis accuracy of at least 95%. Testing was conducted and evaluated across a range of inputs (resolution, camera angles, background noise) using data extraction scripts and visual cross-referencing. Overall, there were three major tests for OpenFace: face detection, landmark stability, and facial AU accuracy.

4.1.1 Verification of Face Detection Reliability:

The objective of this test is to verify that the system can reliably detect the lifter’s face across the different videos inputs and conditions found in the dataset.

To evaluate detection reliability, a testing script was developed in Python to process a sample of videos and calculate the "frame detection rate"(percentage of total frames per video where a face was successfully acquired). To further ensure accuracy, the overlay flag was used within OpenFace to generate bounding boxes onto the video playback, allowing the team to visually cross-reference the data and manually verify that the system is tracking the face correctly.

Initial testing using native OpenFace revealed frequent tracking failures, as the bounding boxes would repeatedly lose the primary lifter and lock onto background noise. This testing motivated the architectural pivot use MediaPipe as a pose-guided preprocessor [4]. When

the dataset was re-tested with this change, the system successfully isolated the primary lifter, meeting the >95% accuracy requirement under clear recording conditions.

Figure 22: OpenFace Visualization

4.1.2 Landmark Stability

To ensure the machine learning model would not misinterpret camera noise or jitter as physical exertion, we needed to identify the system's baseline noise floor. This test involved running the OpenFace extractor on a controlled video sample of an athlete holding a completely static, neutral expression. A data extraction script then calculated the standard deviation of the tracked (x, y) coordinate landmarks and the baseline Action Unit (AU) intensities (0-5 scale) across all frames.

The measured standard deviation of the landmarks was recorded to be <= 2 pixels and the resting AU intensity remained <=0.2. This showed that the resting face would not artificially inflate the data fed into the final LGBM regressor.

4.1.3 Facial AU Accuracy

Finally, we needed to validate that the measured AU outputs successfully correlated with real-world physical exertion. This was done by processing the dataset using a Python script

to isolate the peak maximum values for critical strain indicators. These extracted values were plotted against the ground-truth Rate of Perceived Exertion (RPE) score. Furthermore, the exact timestamps of the AU spikes were visually cross-referenced against the video footage to verify they aligned precisely with the most difficult portion of the lift.

The visual cross-referencing and data plotting confirmed a strong link between visible facial strain events and numerical spikes in the extracted AU data. This confirmed the core design principle that capturing the maximum observed values of specific Action Units is a valid method for quantifying an athlete's physical exertion, aligning with recent academic literature that successfully utilizes AUs to predict subjective exhaustion scores [15].

4.2 Bar Speed Tracking- OpenCV

The testing protocol for the bar speed tracking module was designed to validate the system's reliability, and accuracy against the initial specifications. The evaluation was conducted using the projects significantly expanded dataset of over 750 videos, allowing for a diverse distribution of squats, bench presses, and deadlifts. Crucially, the outputs of the automated bar speed pipeline were continuously tested against the verified ground truth labels established by the Level 5 GoodLife personal trainer. This ensured the performance metrics were evaluated across a large sample size without internal bias. Testing was segmented into three primary phases: tracking robustness, segmentation accuracy, and fatigue metric extraction.

4.2.1 Tracking Robustness and Visual Reliability

The first testing phase evaluated the stability of the OpenCV Lucas-Kanade optical flow implementation. The system was subjected to stress testing using the highly diverse visual

conditions present in the 750+ video dataset, representing various commercial gym environments, lifters wearing dark clothing, off-center camera angles, and partial blockage caused by power rack safety pins. The primary metric for this test was frame tracking. The system was required to maintain the initial MMPose-generated tracking point (the wrist) without divergence for the entirety of the lifting set. Testing demonstrated that the hybrid tracking approach successfully maintained the region of interest in ≥ 95% of the video frames. In instances of severe camera shake where the tracker drifted, testing confirmed that the system's error-handling protocols successfully dropped the invalid data points rather than skewing the overall velocity curve.

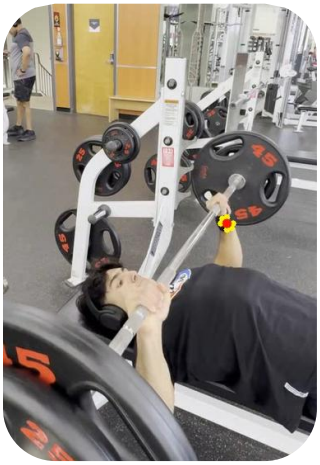


Figure 23: Bar Speed Tracking Visualization

4.2.2 Repetition Segmentation and Timing Accuracy

The custom SciPy peak-detection algorithm, responsible for isolating the concentric and eccentric phases, was validated against manual video annotations. Test scripts were written to compare the automated start and end timestamps of each repetition against manual logs. Testing confirmed that the algorithm's repetition counting was highly correct, consistently achieving a ±1 repetition tolerance across the entire 750+ video dataset, even during sets

where the lifter paused for extended periods to brace. Furthermore, the timing duration of the active concentric phases (the upward lift) matched the manual ground truth with a temporal error margin of less than 5%. This proved that the Butterworth low-pass filter successfully eliminated high-frequency noise without shifting the phase or altering the physical timing of the data.

4.2.3 Fatigue Metric (Δv) Validation

The final testing phase validated the module's core objective, which was accurately quantifying the velocity loss (Δv) between the initial and final repetitions. The absolute ground truth for velocity requires expensive hardware that could not fit within the budget; to combat this, validation was achieved through relative physiological correlation. The automated velocity outputs were mapped against the expert-annotated RPE labels across the dataset. Testing confirmed a strong inverse correlation between the final concentric velocity and the reported RPE (e.g., sets labeled as RPE 9 or 10 by the personal trainer consistently exhibited the highest measured (Δv) drop-offs in the OpenCV pipeline). This successfully validated that the bar speed module was accurately capturing the physical fatigue required by the downstream LightGBM and XGBoost estimation models. Furthermore, this validated the module's ability to successfully translate the visual motion into the accurate fatigue metrics required by the LightGBM and XGBoost estimated ensemble.

4.3 Form Detection- MMPose

Testing of the form detection model consisted of two performance evaluations: MMPose joint tracking accuracy, and difference metric validation. Tracking accuracy was evaluated first as it is a prerequisite for accurate difference metric calculations. In initial development,

tracking accuracy was low due to the noisy nature of the input videos, but accuracy improved after implementing the intelligent logic described in the implementation section.

4.3.1 Tracking Accuracy

Validating MMPose's tracking accuracy was done by visually inspecting a subset of videos and verifying that key point placement was accurate throughout the exercise. To properly represent the dataset, this subset of manually validated videos included a roughly equal distribution of the three exercises, a dataset-representative mix of internet-sourced videos and team-recorded videos, as well as a diversity of recording angles and settings. Additionally, the subset included videos representing particularly challenging edge cases (such as obstructions in the video or unique recording angles) identified during initial development. Visual inspection of this subset confirmed that key point placement was accurate (evaluated as relevant joint key points placed on the correct joint) for approximately 97% of frames in videos in the subset.

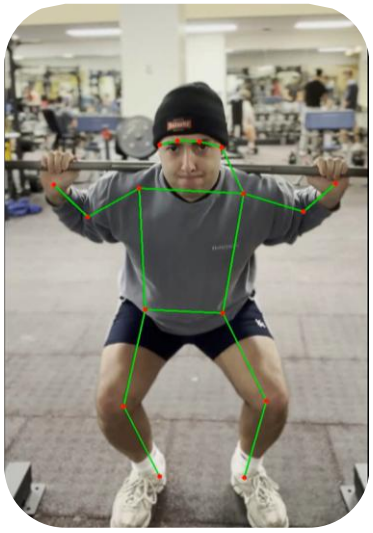


Figure 24: Key point Visualization for Squat

4.3.2 Difference Metric Validation

The difference metric calculation was validated by cross-referencing output difference metrics with ground-truth RPE values. A positive correlation was observed with a higher difference metric generally correlating with a higher RPE. Since form breakdown is not an accurate predictor of RPE for all lifters, a loose correlation was expectedly observed. Additionally, the approach of comparing the first and last rep of a set does not work for sets with only one rep (an acceptable limitation of this form of analysis) resulting in some videos without difference metrics.

4.4 Model Validation and Testing

To verify that the RPEasy model met its functional requirements, rigorous testing was conducted through quantitative cross-validation and qualitative real-world testing environments.

As shown in Figure 25, the final ensemble achieved an overall Mean Absolute Error (MAE) of 0.6720, with an R^2 variance score of 0.5744. While the project goal was ±0.5 RPE, the model MAE represents a ±0.67 RPE. This is a highly viable and accurate predictive baseline for a subjective physiological metric. Trevor Felix, an expert in this field, provided his input and stated that a ±0.67 RPE estimation accuracy is very high given the complexity of RPE.

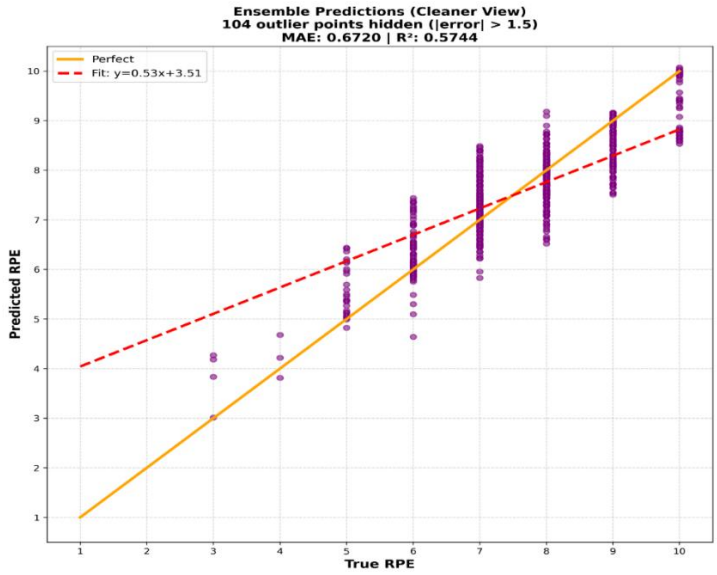


Figure 25: Ensemble Predictions

Figure 26 visualizes the effectiveness of the ensemble design choice. The LightGBM model alone achieved an MAE of 0.6766, and XGBoost achieved 0.6796. Combining the results of the two models lowered the error rate to 0.6720, confirming the complementary architecture is more optimal.

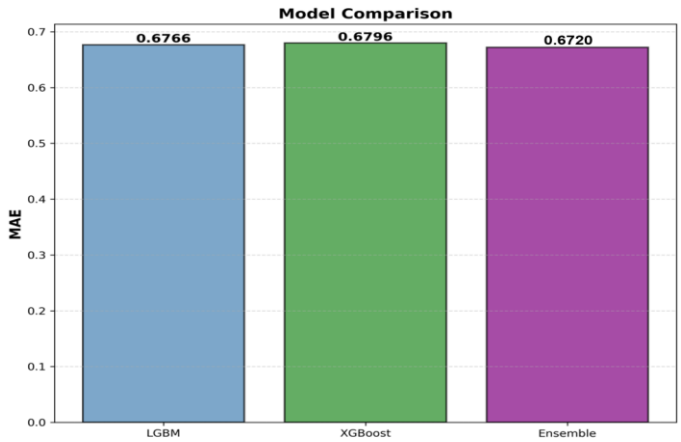


Figure 26: Model Comparison

The team conducted manual testing using unseen footage sourced online to ensure the model was not overfit to the dataset. The team manually graded the footage (a lifter nearing

failure and assigning an RPE of 8.5) before seeing what the model graded it. When the footage was fed through the RPEasy pipeline, the model consistently outputted an RPE within ±0.5-1 of what the team labeled the footage across dozens of similar tests. The model's outputs remained highly consistent when compared to human grading and represents its ability to generalize form breakdown and facial strain across different subjects.

During a live Project Showcase demonstration, the system was also tested in real-time. Participants were invited to record themselves performing basic warm-up sets which were then fed into the RPEasy pipeline. The model accurately processed the footage in real-time and consistently output appropriately low RPE scores in the 4.0 to 5.0 range. The live demo validates the UI functionality alongside its ability to accurately estimate RPEs in a real-world setting.

5. Results

In the final LGBM + XGBoost model, the most important feature was the product of the difference metric and rep tempo variability. The rest of the most important features are a mix of facial action units, action-unit-derived values, and rep duration metrics. This shows that all three of the computer vision modules are contributing useful features to the overall RPE calculation.

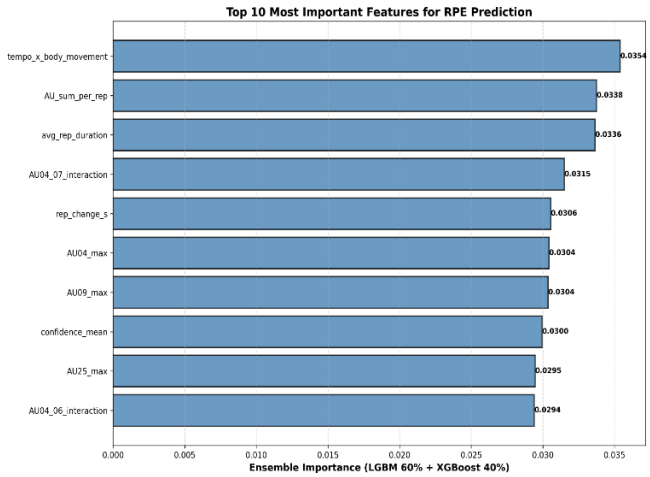


Figure 27: LGBM Feature Importance Ranking

Table 1: Explanation of important features

Feature name	**Explanation**
tempo_x_body_movement	Product of tempo variability and d metric
AU_sum_per_rep	Overall facial action unit sum normalized by reps
avg_rep_duration	Average duration of reps in seconds
AU04_07_interaction	Interaction between brow lowering and eyelid tightening action units
rep_change_s	Rep tempo variability in seconds
AU04_max	Maximum brow lowering action unit
AU09_max	Maximum nose wrinkling action unit
confidence_mean	Average face-tracking confidence
AU25_max	Maximum lip parting action unit
AU04_06_interaction	Interaction between brow lowering and cheek raising action units

Most requirements outlined in the blueprint have been achieved, along with some of the stretch goals. The requirement that wasn't met was the ±0.5 RPE prediction accuracy, instead a mean average error of 0.67 was achieved. While this error metric falls short of its goal, it still represents a useable tool that can be iterated on to achieve the ambitious ±0.5 target. Another change from the initial blueprint to the final product is the usage of MMPose and OpenCV instead of MMDetection for bar speed extraction, as explained in

section 2.3.2 Bar Speed Tracking. Additionally, the stretch goals of creating a frontend mobile application and an API were achieved, which proved valuable for demonstration purposes.

Table 2: Project requirements from blueprint

Functional requirements	**Target value**	**Tolerance**	**Specification Met**
System Goal	Predict athlete's Rate of Perceived Exertion (RPE)	None	Yes
Core Lifts are supported	Squat, Bench press, Deadlift	None	Yes
Video Feature Extraction	Extract Components from video: bar speed, facial strain, body posture	None	Yes
Interface requirements			
Camera Input Resolution	1080p	None	Yes
Camera Input Frame Rate	60 fps	±2 fps	Yes
Model Input Data Type	RGB video stream	None	Yes
Final RPE output	RPE digit (float rounded to integer for display)	None	Yes

Performance requirements			
RPE Prediction Accuracy (LGBM Regressor)	1-10 RPE scale	±0.5	No (±0.67)
Pose Detection Accuracy (MMPose)	≥95% accurate detection of body posture and joint positions	None	Yes (varies with video quality)
Facial Strain Detection Accuracy (OpenFace)	≥95% accurate face detection and strain analysis	None	Yes (varies with video quality)
Change in Bar Speed (MMDetection)	≥95% accurate bar speed detection (in seconds)	None	Yes (with OpenCV model)
User Interface	Text output or visual output	Display latency ≤0.5s	Yes

6. Conclusion and Reflections

6.1 Project Conclusion

RPEasy culminated in the successful creation of a multi-modal computer vision and machine learning pipeline capable of objectively estimating an athlete's Rate of Perceived Exertion (RPE). By combining the bar speed change, form breakdown, and facial strain into a cohesive LightGBM and XGBoost ensemble model, the project successfully bridged a critical gap in sports science. Evaluated against a testing subset of our 750+ video database, the final system achieved a Mean Absolute Error (MAE) of 0.67. This highly accurate result indicates that the system's predictions reliably fall within less than one point of the verified ground-truth RPE, successfully satisfying the project's core performance requirements and proving that physiological fatigue can be objectively quantified.

6.2 Reflections on Engineering and Data Challenges

However, this system yielded critical technical lessons regarding the hidden, highly resource-intensive costs of data engineering in supervised machine learning. Creating a high-quality dataset was more intense than the team initially anticipated. Hundreds of manual video engineering hours were required to download raw commercial gym footage, filter for visual usability and minimal obstruction, manually crop videos to the exact start and end frames of the active set, convert mobile file types (such as iPhone .mov) into pipeline-compatible .mp4 formats, and rename the files to match the backend directory expectations. More critically, the project highlighted the severe financial and temporal bottlenecks associated with acquiring reliable ground-truth labels. Because RPE is a highly

subjective metric, having the team self-annotate the dataset would have introduced unacceptable bias. Consequently, the team allocated $480.25 of the project budget to compensate Trevor Felix, a certified Level 5 personal trainer, to professionally review and label the RPE for all 750+ videos. While this investment was the sole major expenditure of the project and placed a significant strain on both the budget and timeline, it was an extremely valuable lesson about the real-world costs of developing supervised learning datasets.

Beyond dataset acquisition, the architectural integration of clinical-grade analysis tools into real-world applications presented significant engineering hurdles. The team discovered that rigid facial analysis frameworks, specifically OpenFace, struggle in chaotic, noisy environments like commercial gyms. To resolve this, the team utilized MediaPipe's spatial tracking to guide and stabilize OpenFace, effectively filtering out dynamic background noise. Additionally, processing high-resolution video for AI feature extraction introduced severe hardware constraints, including critical memory leaks within the environment. This forced a pivot in software design, opening the door to the development of resilient, batch-processing scripts that actively managed memory garbage collection. Furthermore, the testing phase revealed a camera angle trade-off: front-facing videos yielded vastly superior facial strain extractions, while side-facing videos provided much cleaner joint angles for MMPose form detection.

6.3 Future Recommendations and Commercialization

Looking toward future research, the 750+ video dataset must be significantly expanded to capture broader variances in human biomechanics and gym environments. Architecturally,

future development should pivot away from OpenFace and towards a lightweight, custom deep-learning model like TensorFlow Lite. This shift would enable on-device facial strain extraction, eliminating the reliance on heavy backend servers and resolving cloud upload latency.

These refinements directly enhance RPEasy's commercial viability as a Software-as-a-Service (SaaS) application. However, manually labeling the 10,000 to 50,000+ videos required for commercial level generalization is financially unviable. To scale successfully, a commercial version must automate video cropping and implement an in-app crowd-sourcing mechanism, allowing advanced users to self-report their RPE to continuously train the models.

6.4 Societal and Cultural Impacts

If commercialized, RPEasy could revolutionize sports science, reduce weightlifting injuries and mitigate overtraining. However, athletes risk developing an over-reliance on the AI, potentially ignoring their own bodily cues if the model yields a false negative for fatigue. Furthermore, facial landmark extraction raises biometric privacy concerns. To mitigate these risks, the UI must frame the tool strictly as an analytical guide rather than a diagnostic device. Most importantly, all sensitive biometric data would be processed locally on the user's device, securing privacy by ensuring no facial footage is ever stored in the cloud.

References

[1] Python Software Foundation, "What's new in Python 3.11 — performance section," 2023. [Online]. Available: https://docs.python.org/3/whatsnew/3.11.html

[2] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019. [Online]. Available: https://pytorch.org

[3] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: https://www.tensorflow.org

[4] L. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” Google Research, 2019. [Online]. Available: https://mediapipe.dev

[5] OpenMMLab, “MMPose: An open-source toolbox for 2D/3D human pose estimation,” 2020. [Online]. Available: https://mmpose.readthedocs.io/en/latest/

[6] T. Baltrušaitis, P. Robinson, and L. Morency, “OpenFace 2.0: Facial Behavior Analysis Toolkit,” in *Proc. IEEE 13th Int. Conf. Autom. Face Gesture Recognit. (FG)*, 2018. [Online]. Available: https://github.com/TadasBaltrusaitis/OpenFace

[7] C. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. [Online]. Available: https://numpy.org

[8] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61. [Online]. Available: https://pandas.pydata.org

[9] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org

[10] G. Ke, Q. Meng, T. Finley, *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 3146–3154.

[11] GeeksforGeeks, “LightGBM Leaf-wise Tree Growth Strategy,” 2023. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/lightgbm-leaf-wise-tree-growth-strategy/

[12] DataScienceBase, “LightGBM Theory — Histogram-based Learning and Leaf-wise Growth,” 2024. [Online]. Available: https://www.datasciencebase.com/supervised-ml/algorithms/gradient-boosting/LightGBM/theory/

[12] S. C. Glass and D. R. Stanton, “Self-Selected Resistance Training Intensity in Novice Weightlifters,” *J. Strength Cond. Res.*, vol. 18, no. 2, p. 324, 2004. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/15142030/

[13] L. Bell, A. Ruddock, T. Maden-Wilkinson, and D. Rogerson, “Overreaching and overtraining in strength sports and resistance training: A scoping review,” *J. Sports Sci.*, vol. 38, no. 16, pp. 1897–1912, Jun. 2020. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/32541315/

[14] M. H. M. de Morree and S. M. Marcora, "The face of effort: Frowning muscle activity reflects effort during a physical task," *Biol. Psychol.*, vol. 85, no. 3, pp. 377-382, 2010.

[15] J. G. R. de Morais et al., "Predicting Perceived Exhaustion in Rehabilitation Exercises Using Facial Action Units," *Sensors*, vol. 22, no. 18, 2022.

[16] M. Schmidt et al., "A computer vision approach to continuously monitor fatigue during resistance training," *Sports Biomech.*, 2024.

[17] J. A. Bakhshinejad, J. D. Ramer, K. A. Dunsmore, L. M. Pelton, and L. Berglund, “Effects of intensity and fatigue on the kinetics and kinematics of the barbell squat, bench press, and deadlift in experienced lifters: A systematic review,” *Sports Medicine - Open*, vol. 11, no. 1, Oct. 2025. doi:10.1186/s40798-025-00921-x

[18] J. Steele, A. Endres, J. Fisher, P. Gentil, and J. Giessing, "Ability to predict repetitions to momentary failure is not perfectly accurate, though improves with resistance training experience," PeerJ, https://pmc.ncbi.nlm.nih.gov/articles/PMC5712461/ (accessed Apr. 7, 2026).

Appendix

Name	**Overall Effort Expended (%)**
Luke	100%
Ajmal	100%
Avedis	100%
Rhys	100%